

Práctica 6

1. ¿Cuál es el puerto por defecto que se utiliza en los siguientes servicios? Web / SSH / DNS / Web Seguro / POP3 / IMAP / SMTP

- HTTP (web) utiliza el puerto 80, pero HTTPS (web seguro) utiliza el 443 [TCP]
- SSH utiliza el puerto 22 ([RFC 4253](#)) [TCP]
- DNS utiliza el puerto 53 ([RFC 1035](#)) [TCP y UDP]
- POP3 utiliza el puerto 110 [TCP]
- IMAP utiliza el puerto 143 ([RFC 3501](#)) [TCP]
- SMTP utiliza el puerto 25 ([RFC 5321](#)) [TCP]
[Corregido]

Investigue en qué lugar en Linux y en Windows está descrita la asociación utilizada por defecto para cada servicio

- En Linux por lo general esto se lo puede encontrar en el archivo **/etc/services**, donde a cada servicio (de protocolo) mencionado se le asigna un puerto y un protocolo de transporte
- En Windows también hay un archivo que nos muestra este mapeo (**C:\Windows\System32\drivers\etc\services**). En la siguiente página de la documentación oficial de Windows se detalla: <https://learn.microsoft.com/en-us/troubleshoot/windows-server/networking/service-overview-and-network-port-requirements>

[Corregido]

2. Investigue qué es multicast. ¿Sobre cuál de los protocolos de capa de transporte funciona? ¿Se podría adaptar para que funcione sobre el otro protocolo de capa de transporte? ¿Por qué?

- El multicast es una forma de comunicación entre hosts, donde un host emisor puede mandar mensajes que reciben mas de un host receptor a la vez.

Comunmente funcionan sobre UDP, ya que UDP es un protocolo que no hace otra cosa que enviar un paquete a un destino

"Una **conexión TCP casi siempre es una conexión punto a punto**, es decir, entre un único emisor y un único receptor. La "multidifusión", **la transferencia de datos desde un emisor a muchos receptores en una única operación de envío, no es posible con TCP**. Con TCP, dos hosts son compañía y tres multitud." (Kurose, p. 191)

Se podría trabajar utilizando el protocolo TCP, pero el resultado sería mucho mas ineficiente que si se decidiera utilizar UDP, ya que TCP es un protocolo de transporte que establece conexiones de un host a otro host, y no de un host a multiples hosts. En cualquier caso, se podría emular una conexión uno a muchos con TCP, pero eso requeriría el tiempo necesario para realizar N conexiones, requeriría enviar N veces el mismo paquete por red cada vez que se envíe un paquete, requeriría incluir las N direcciones de los multiples receptores dentro del paquete, y demás cuestiones.

"En el caso de la comunicación unicast, la dirección IP del receptor (el destino) es llevado por cada datagrama IP unicast, e identifica al único destinatario. Pero en el caso del multicasting, ahora tenemos varios receptores. ¿Tiene sentido que por cada datagrama multicast lleve dentro todas las direcciones de los receptores? Mientras que este enfoque podría funcionar con un pequeño número de recipientes, no escalaría bien en el caso de que hayan miles de receptores; la cantidad de información de direccionamiento incluida en el datagrama sobrepasaría por mucho la información útil que se trae en el payload. La identificación explícita de los receptores por el emisor también requeriría que el emisor conozca quienes son y cuales son las direcciones de cada receptor."

[http://www2.ic.uff.br/~michael/kr1999/4-network/4_08-mcast.htm]

ADDENDUM: "A nivel de transporte el protocolo adecuado para enviar mensajes broadcast o multicast es el UDP." [Barbieri, p. 10]

[Corregido]

3. Investigue cómo funciona el protocolo de aplicación FTP teniendo en cuenta las diferencias en su funcionamiento cuando se utiliza el modo activo de cuando se utiliza el modo pasivo ¿En qué se diferencian estos tipos de comunicaciones del resto de los protocolos de aplicación vistos?

- El protocolo FTP establece **dos** conexiones entre el cliente y el servidor: una que sirve para el **pasaje de comandos**, y otra que sirve para la **transmisión de datos**. Cuando se utiliza el modo activo, el cliente establece la **conexión de control** (la de comandos), y el servidor se encarga de establecer la **conexión de datos**. En el modo pasivo, el cliente tiene la tarea de establecer ambas conexiones.

A mi entender y con lo que recuerdo ahora, estos tipos de comunicaciones se diferencian del resto de protocolos en que se utilizan dos conexiones para llevar a cabo cada parte de manera separada, en vez de utilizar una sola conexión que primero establezca el vínculo entre cliente y servidor, y luego haga la tarea, como transferir datos.

[Corregido]

4. Suponiendo Selective Repeat; tamaño de ventana 4 y sabiendo que E indica que el mensaje llegó con errores. Indique en el siguiente gráfico, la numeración de los ACK que el host B envía al Host A.

En orden, serían los siguientes: ACK 1, ACK 2, ACK 4, ACK 5, ACK 3 (y es recién en este momento cuando se desplaza la ventana del receptor y la del emisor)

[Corregido]

5. ¿Qué restricción existe sobre el tamaño de ventanas en el protocolo Selective Repeat?

a ventana no puede avanzar si tiene un segmento no confirmado.

[Corregido]

6. De acuerdo a la captura TCP de la siguiente figura, indique los valores de los campos borroneados

SOURCE	DESTINATION	INFO
		[SYN] Seq=3933822137
172.20.1.1	172.20.1.100	41749 > vce [ACK] Seq=3933822138 Ack=1047471502 (1047471501 + 1)

[Corregido]

7. Dada la sesión TCP de la figura, completar los valores marcados con un signo de interrogación

1. SEQ=1, ACK=1
2. SEQ=1, ACK=8
3. SEQ=1, ACK=17
4. SEQ=1, ACK=22
5. SEQ=22, ACK=1
6. SEQ=1, ACK=23
7. SEQ=23, ACK=2

[Corregido]

8. ¿Qué es el RTT y cómo se calcula? Investigue la opción TCP timestamp y los campos TSval y TSecr.

El RTT es el Round-Trip Time, y se calcula restando el TSecr con el TSval.

"La opción Timestamp le permite al emisor colocar dos valores de 4 bytes de marca de tiempo en cada segmento. El receptor establece estos valores en el reconocimiento, permitiendo que el emisor calcule un estimado del RTT de conexión por cada ACK recibido. Cuando se utiliza la opción de Timestamp, el emisor coloca un valor de 32 bits en el campo de Valor del Timestamp (**TSval**) en la primera parte del TSOPT (opción Timestamp), y el receptor copia este valor y lo pone en el campo del Timestamp Echo Retry (**TSecr**)."
[Stevens, p. 608 - 609]

"El TSval contiene el valor actual del reloj timestamp del TCP que envía la opción. El **TSecr** solo es válido si el bit de ACK está activado en el encabezado del TCP; si es válido, copia el valor del timestamp enviado por el TCP remoto en el campo **TSval**. Cuando **TSecr** no es válido, su valor debe ser 0."
[RFC 1323, p. 13]

[Consultar]

9. Para la captura tcp-captura.pcap, responder las siguientes preguntas.

a. ¿Cuántos intentos de conexiones TCP hay?

- Está la **primera conexión**, que ocurre entre los puertos 46907 y el 5001 [paquetes 1 - 960]
- luego está la **segunda**, entre los puertos 45670 y el 7002 (en la que se recibe un paquete con flag RST) [961 - 962]
- la **tercera** entre los puertos 45670 y 7002 [963 - 964]
- la **cuarta conexión** entre los puertos 46910 y 5001 (que es exitosa) [967 - 980]
- la **quinta conexión** entre los puertos 54424 y el 9000 (también exitosa pero que termina mal) [981 - 1105]
- la **sexta conexión** entre los puertos 54425 y el 9000 (exitosa) [1106 - 1671]

b. ¿Cuáles son la fuente y el destino (IP:port) para c/u?

- Para la primera, la fuente es 10.0.2.10:46907 y el destino 10.0.4.10:5001
- Para la segunda, la fuente es 10.0.2.10:45670 y el destino 10.0.4.10:7002
- Para la tercera, la fuente es 10.0.2.10:45671 y el destino 10.0.4.10:7002
- Para la cuarta, la fuente es 10.0.2.10:46910 y el destino 10.0.4.10:5001
- Para la quinta, la fuente es 10.0.2.10:54424 y el destino 10.0.4.10:9000
- Para la sexta, la fuente es 10.0.2.10:54425 y el destino 10.0.4.10:9000

c. ¿Cuántas conexiones TCP exitosas hay en la captura? ¿Cómo diferencia las exitosas de las que no lo son? ¿Cuáles flags encuentra en cada una?

Conexiones TCP exitosas hay 3 nada mas. Las diferencio a esas tres de las demás ya que establecen una conexión mediante el Triple Handshake, y la cierran mediante el 4-Way Close. En las exitosas se puede ver como, en el principio, se mandan los paquetes de inicio SYN, SYN-ACK y ACK, y al final se mandan los de cierre, que son los que tienen FIN y ACK.

d. Dada la primera conexión exitosa responder:

i. ¿Quién inicia la conexión?

La conexión la inicia el host con dirección IP 10.0.2.10 y puerto 46907

ii. ¿Quién es el servidor y quién el cliente?

El servidor es el host con dirección 10.0.4.10 y puerto 5001, y el cliente es el que mencioné la otra vez ¿O se me pide fijarme en otra cosa, por ejemplo si se incrementa el número de secuencia propio o no?

iii. ¿En qué segmentos se ve el 3-way handshake?

En los segmentos que van del 3 al 5

iv. ¿Cuáles ISNs se intercambian?

El ISN del cliente que sería el número 2218428254, y el del servidor que sería el 1292618479

v. ¿Cuál MSS se negoció?

Entre los dos se aceptó un MSS de 1460 bytes. Esto se encuentra en la sección Opcion del segmento TCP.

vi. ¿Cuál de los dos hosts envía la mayor cantidad de datos (IP:port)?

El que manda la mayor cantidad de datos es el host cliente, y esto lo digo porque su SEQ es el único que incrementa a lo largo de la conexión

e. Identificar primer segmento de datos (origen, destino, tiempo, número de fila y número de secuencia TCP). → ¿Por cada conexión?

a. ¿Cuántos datos lleva?

El primer segmento de datos (de origen 10.0.2.10:46907, destino 10.0.2.10:5001, tiempo 0.151826, número de fila 6 y SEQ (relativo) 1) lleva 24 bytes de datos.

b. ¿Cuándo es confirmado (tiempo, número de fila y número de secuencia TCP)?

La recepción de los datos es confirmado en el siguiente paquete, tiempo 0.151925, num. fila 7, num. sec 1.

c. La confirmación, ¿qué cantidad de bytes confirma?

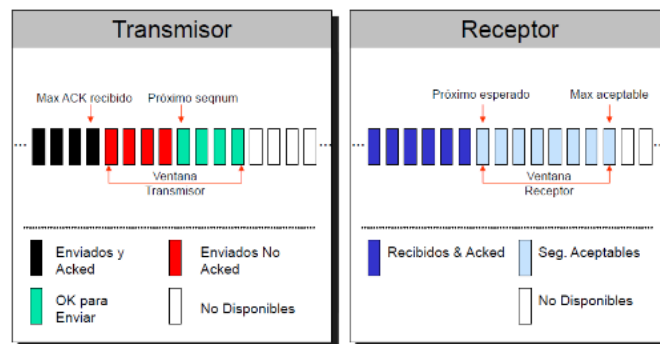
Confirma 24 bytes, ya que el número de secuencia pasa de ser 1 a ser 25

- f. ¿Quién inicia el cierre de la conexión? ¿Qué flags se utilizan? ¿En cuáles segmentos se ve (tiempo, número de fila y número de secuencia TCP)? →
¿Por cada conexión?

El que inicia el cierre de la conexión es el host cliente. Se utilizan los flags FIN, PSH, y ACK. El cierre se ve entre los segmentos número 958 y 960, tiempos 75.09016, 75.091719, 75.247457, y números de secuencia 786289, 1, 786458, y 2

[Consultar]

10. Responda las siguientes preguntas respecto del mecanismo de control de flujo



- a. ¿Quién lo activa? ¿De qué forma lo hace?

Quien lo activa es, en realidad, cada proceso desde su lado de la conexión TCP. Esto se hace manteniendo y enviando unas variables desde el **lado receptor** que indican **cuantos bytes del flujo fueron leídos**, y **cuantos bytes del mismo flujo fueron recibidos**, y desde el **lado emisor** manteniendo y enviando unas variables que indican **cuantos bytes fueron enviados**, y **cuantos fueron reconocidos**.

"¿Cómo utiliza la conexión la variable **VentRecepcion** para proporcionar el servicio de control de flujo? El host B dice al host A la cantidad de espacio disponible que hay en el buffer de la conexión almacenando el valor actual de **VentRecepcion** en el campo ventana de recepción de cada segmento que envía a A. Inicialmente, el host B establece que $VentRecepcion = BufferRecepcion$." [Kurose, p. 208]

b. **¿Qué problema resuelve?**

El problema que resuelve es el de **evitar el desbordamiento de los buffers de recepción**, osea, el de que no llegue mas información de la que se puede mantener desde un lado de la conexión.

"Si la aplicación es relativamente lenta en lo que respecta a la lectura de los datos, el emisor puede fácilmente desbordar el buffer de recepción de la conexión enviando muchos datos demasiado rápidamente. TCP proporciona un servicio de control de flujo a sus aplicaciones para eliminar la posibilidad de que el emisor desborde el buffer del receptor" [Kurose, p. 207]

c. **¿Cuánto tiempo dura activo y qué situación lo desactiva?**

No entiendo a qué se refiere o no encuentro una respuesta. Consultar
[Consultar]

11. Responda las siguientes preguntas respecto del mecanismo de control de congestión

a. **¿Quién activa el mecanismo de control de congestión? ¿Cuáles son los posibles disparadores?**

Es el **emisor** mismo quien "activa" el mecanismo. El método empleado por TCP consiste en que cada emisor limite la velocidad a la que transmite el tráfico a través de su conexión en función de la congestión de red percibida. Si un emisor TCP percibe que en la ruta entre él mismo y el destino apenas existe congestión, entonces incrementará su velocidad de transmisión; por el contrario, si el emisor percibe que existe congestión a lo largo de la ruta, entonces reducirá su velocidad de transmisión

b. **¿Qué problema resuelve?**

El problema que se resuelve es el de que se estén enviando mas paquetes de los que se pueden procesar de manera óptima por una conexión.

c. **Diferencie slow start de congestion-avoidance.**

El algoritmo de **arranque lento** se utiliza en la primera fase del control de congestión, y el de evitación de la congestión en la segunda. Con el primer algoritmo, la cantidad máxima que se puede enviar de datos se va incrementando en 1 MSS mas por cada reconocimiento del primer segmento transmitido (en esa serie de segmentos enviados), haciendo que la velocidad

de transmisión de TCP crezca exponencialmente. Con el segundo, se incrementa el valor de VentCongestion solamente en un MSS por cada RTT.
[Consultar]

12. Para la captura udp-captura.pcap, responder las siguientes preguntas

a. ¿Cuántas comunicaciones (srcIP, srcPort, dstIP, dstPort) UDP hay en la captura?

En la captura hay 10 comunicaciones:

- La primera es la de (10.0.2.10:0 / 10.0.3.10:8003)
- La segunda es la de (10.0.2.10:9000 / 10.0.3.10:13)
- (10.0.2.10:9004 / 10.0.3.10:13)
- (10.0.2.10:9004 / 10.0.3.10:4555)
- (10.0.2.10:9004 / 10.0.3.10:9045) [79 - 82]
- (10.0.2.10:9004 / 1.1.1.1:9045) [83 - 725]
- (10.0.0.2:xxxx / 10.0.2.10:9004) → (Consultar cual sería el puerto de 10.0.0.2 — 726)
- (10.0.2.10:53300 / 10.0.4.10:9045)
- (10.0.2.10:59053 / 10.0.4.10:8003)
- (10.0.2.10:8003 / 10.0.4.10:8003) [734 - 3001]

b. ¿Cómo se podrían identificar las exitosas de las que no lo son?

Se podrían calificar en cuanto pudieron intercambiarse más de un mensaje entre ellas

c. ¿UDP puede utilizar el modelo cliente/servidor?

Es posible, pero no sería óptimo. Aunque puede ser que el proceso servidor le responda al proceso cliente lo que sea que le haya solicitado, también es posible que no lo haga, y a menos que se le provea a la comunicación de una confirmación o protocolo alguno para asegurarse de que lleguen los

mensajes, UDP no sería muy útil para la comunicación cliente/servidor. →
[Consultar](#)

d. **¿Qué servicios o aplicaciones suelen utilizar este protocolo? ¿Qué requerimientos tienen?**

Los servicios o aplicaciones que suelen utilizar este protocolo son los que están relacionados con la transmisión de multimedia en tiempo real, como los servicios de Streaming, las aplicaciones de comunicaciones, y los sitios como Twitch o Kick. Estos servicios suelen utilizar este protocolo debido a que no les interesa que se pueda llegar a perder información en el camino, si no que priorizan que llegue rápido y a tiempo, sus requerimientos son estos:

- **Tiempo de respuesta rápido**
- **Confiabilidad poca**
- **Seguridad poca o mucha (dependiendo de cuanto se quiera proteger una comunicación o multimedia)**
- **Ancho de banda optimizado**

[\[Consultar\]](#)

e. **¿Qué hace el protocolo UDP en relación al control de errores?**

El protocolo UDP implementa solo un mecanismo para el control de errores, que no es obligatorio ni se lo requiere en varios casos, el *Checksum*:

"El largo del datagrama UDP puede ser de un número impar de bytes, mientras que el algoritmo de checksum agrega words de 16-bits. El procedimiento para UDP es el de agregar un pad byte (virtual) de 0 al final de los datagramas de longitud impar, solo para la computación y verificación del checksum. Este pad byte no se transmite en realidad y por lo tanto yo lo llamo "virtual"." [\[Stevens, p. 476\]](#)

Si el checksum se verifica y se determina que no es válido, el paquete se descarta.

"Si el emisor computa el checksum y el receptor detecta un error en el checksum, el paquete UDP es descartado silenciosamente." [\[Stevens, p. 477\]](#)

[\[Consultar\]](#)

- f. **Con respecto a los puertos vistos en las capturas, ¿observa algo particular que lo diferencie de TCP?**

Las únicas cositas diferentes que noto, es que se llegan a utilizar puertos menores a 5000

[Consultar]

- g. **Dada la primera comunicación en la cual se ven datos en ambos sentidos (identificar el primer datagrama):**

- i. **¿Cuál es la dirección IP que envía el primer datagrama?, ¿desde cuál puerto?**

La dirección IP del que envía el primer datagrama es 10.0.2.10, y lo envía desde el puerto 9004

- ii. **¿Cuántos datos se envían en un sentido y en el otro?**

En dirección **emisor** → **receptor** se envían 2 paquetes

En dirección **receptor** → **emisor** se envían otros 2 paquetes

[Consultar]

13. Dada la salida que se muestra en la imagen, responda los ítems debajo.

- **Suponga que ejecuta los siguientes comandos desde un host con la IP 10.100.25.90. Responda qué devuelve la ejecución de los siguientes comandos y, en caso que corresponda, especifique los flags.**

- a. **hping3 -p 3306 -udp 10.100.25.135**

Debería devolver un mensaje por ICMP (¿quizás el de Port Unreachable?), ya que el protocolo que se está utilizando en este caso es UDP.

- b. **hping3 -S -p 25 10.100.25.135**

Le envía un paquete TCP con flag SYN activado a la dirección 10.100.25.135:25. **¿Tendrá algo que ver con la dirección local :1:25?**

- c. **hping3 -S -p 22 10.100.25.135**

Según la imagen de la salida, ya hay una conexión que utiliza la dirección 10.100.25.135:22, pero igual no sé que significa o implica esto, porque

probé recién en mi máquina recrear la situación de enviarle un paquete a una dirección IP local que se encuentra en estado LISTEN, y no me devolvió nada, salvo que se perdieron todos los paquetes enviados.

d. **hping3 -S -p 110 10.100.25.135**

No sé que tendría que devolver, pero supongo que al no estar escuchando el puerto solo se tendría que ver como se perdieron todos los paquetes enviados también. Eso o se envía un paquete con el flag RST activado.

[Consultar]

- **¿Cuántas conexiones distintas hay establecidas? Justifique**

Hay 3 conexiones distintas establecidas:

- (127.0.0.1:3306 -- 127.0.0.1:34338) ↔ (127.0.0.1:34338 -- 127.0.0.1:3306)
- (127.0.0.1:48717 -- 127.0.0.1:3306) ↔ (127.0.0.1:3306 -- 127.0.0.1:48717)
- (10.100.25.135:22 -- 127.0.0.1:3306)

Por mas que aparezcan 5 conexiones en estado ESTABLISHED, 2 de ellas pertenecen a la misma conexión que otras dos debido a que sus direcciones y puertos fuentes y destino son los mismos, nada mas que cambiados de lado.

[Consultar]